

The solution: output_key and state templating

How it works:

- 1 Writing: Use `output_key` to save an agent's output to state
- 2 Reading: Use `{key}` in instructions to read from state

Python

```
from google.adk.agents import LlmAgent, SequentialAgent

# Agent 1: Writes to state
researcher = LlmAgent(
    model='gemini-2.5-flash',
    name='researcher',
    output_key='research_data', # Saves output to state['research_data']
    instruction='Research the given topic and provide key findings'
)

# Agent 2: Reads from state
writer = LlmAgent(
    model='gemini-2.5-flash',
    name='writer',
    instruction='Write an article using this research: {research_data}' # Reads from state
)

# Sequential workflow
root_agent = SequentialAgent(
    name='content_pipeline',
    sub_agents=[researcher, writer]
)
```

What happens:

- 1 User asks about a topic
- 2 Researcher runs, output saved to `state['research_data']`
- 3 Writer runs, `{research_data}` replaced with actual research
- 4 Writer uses the research to create content

Reference: [ADK Docs - State-Based Communication](#)



Core concept: State-based communication

How output_key works

When you set `output_key` on an agent:

```
Python
agent = LlmAgent(
    name='my_agent',
    output_key='my_output', # The key name
    instruction='Do something...'
)
```

The agent's response is automatically saved to session state:

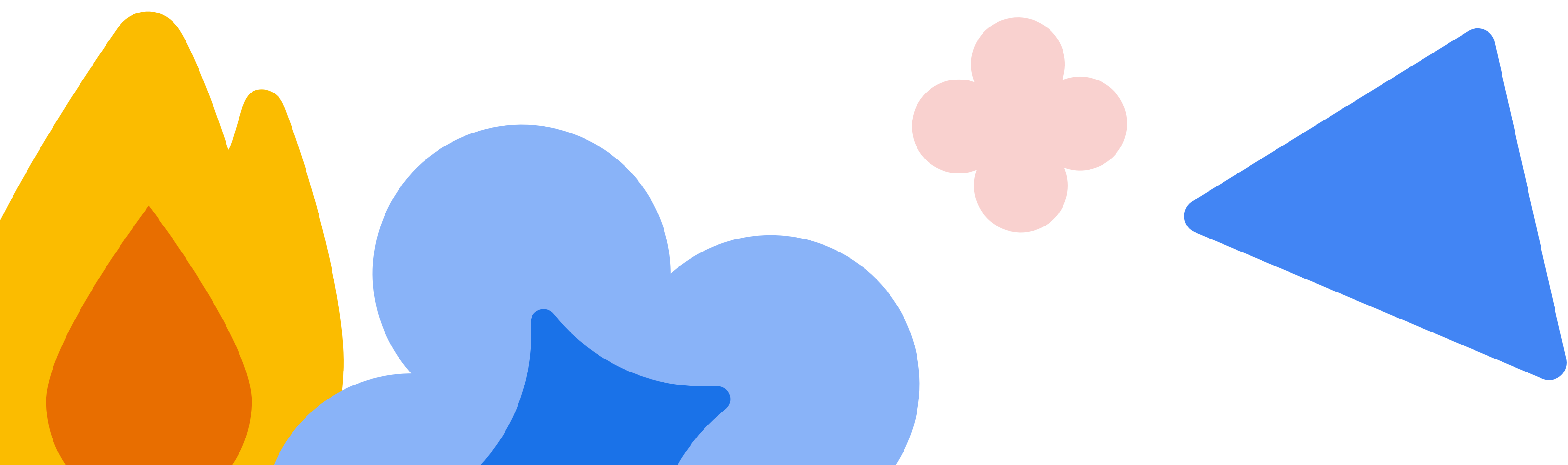
```
Python
agent = LlmAgent(
    instruction='Use this data: {my_output}' # Reads from state
)
```

How state templating works

When you use `{key}` in an instruction:

```
Python
agent = LlmAgent(
    instruction='Use this data: {my_output}' # Reads from state
)
```

ADK replaces `{key}` with the value from state before the agent runs.



Complete pattern

```
Python
# Step 1: Agent writes
intake = LlmAgent(
    name='intake',
    output_key='customer_request', # Write here
    instruction='Extract the customer request'
)

# Step 2: Agent reads
processor = LlmAgent(
    name='processor',
    output_key='processed_data', # Write here
    instruction='Process: {customer_request}' # Read from Step 1
)

# Step 3: Agent reads
responder = LlmAgent(
    name='responder',
    instruction='Respond based on: {processed_data}' # Read from Step 2
)

# Chain them together
pipeline = SequentialAgent(
    name='pipeline',
    sub_agents=[intake, processor, responder]
)
```

Key points:

- Each agent can write with **output_key**
- Each agent can read with **{key}** templating
- Data flows through the pipeline via state
- All agents share the same session state

The following diagram illustrates how data flows between agents through shared session state. Each agent writes to state using **output_key**, and the next agent reads that data using **{key}** templating:

```
None
graph LR
    A[Researcher] -->|output_key='research'| STATE[(State)]
    STATE -->|"{"research}"| B[Writer]
    B -->|output_key='draft'| STATE
    STATE -->|"{"draft}"| C[Editor]

    style STATE fill:#fff3cd
    style A fill:#e1f5ff
    style B fill:#e1f5ff
    style C fill:#e1f5ff
```

State (yellow) acts as the central data store. Agents (blue) write outputs and read inputs through this shared state.

Other communication patterns (brief overview)

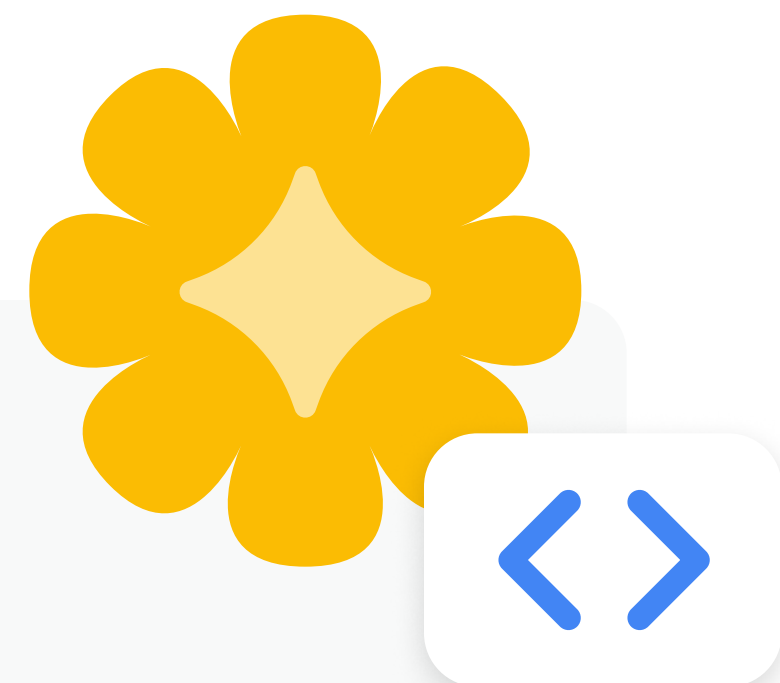
While `output_key` is the primary pattern, ADK supports two other approaches:

LLM-driven delegation

For dynamic routing where you can't predetermine which agent should handle a request:

Python

```
# The LLM decides which sub-agent to use
root_agent = LlmAgent(
    name='coordinator',
    instruction='Route billing questions to
billing_specialist, tech questions to tech_specialist',
    sub_agents=[billing_specialist, tech_specialist] # LLM
chooses
)
```



This is useful when routing depends on understanding the user's request. The LLM reads agent descriptions and decides which specialist to delegate to.

AgentTool (explicit invocation)

For explicit control over when to call another agent:

Python

```
# The LLM decides which sub-agent to use
root_agent = LlmAgent(
    name='coordinator',
    instruction='Route billing questions to
billing_specialist, tech questions to tech_specialist',
    sub_agents=[billing_specialist, tech_specialist] # LLM
chooses
)
```

You already saw this in part 1. It gives you fine-grained control over invocation.



When to use each

Pattern	Use When
output_key + {key}	Data flowing through pipelines (most common)
sub_agents (LLM routing)	Dynamic routing based on request content
AgentTool	Explicit control, specialized subtasks

For this lesson, focus on `output_key` - it's the most common and essential pattern.

Hands-on example

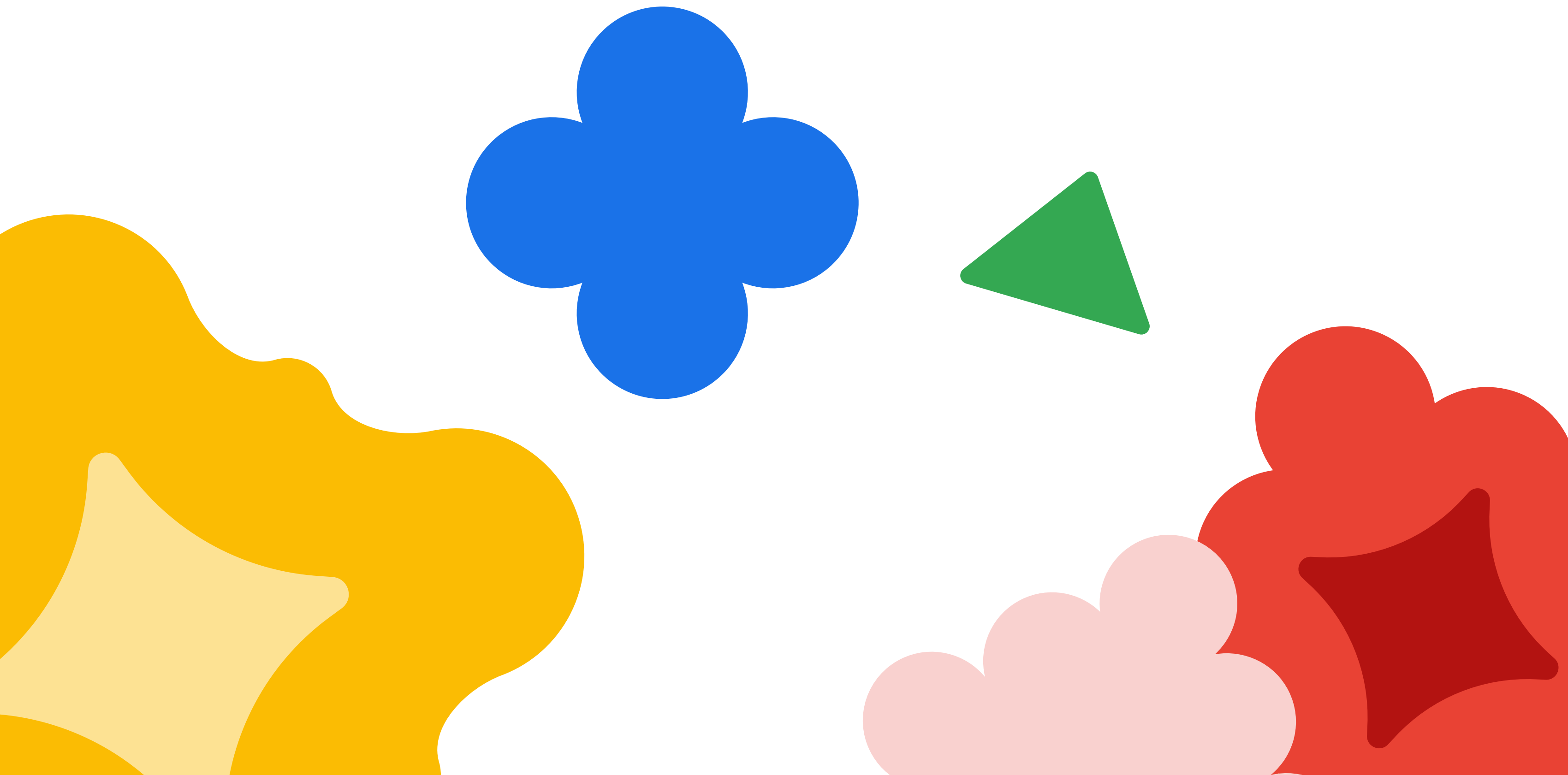
Building a research-to-article pipeline

What you'll build: A pipeline where a researcher passes data to a writer.

Step 1: Create the project

Shell

```
adk create research_pipeline
cd research_pipeline
```



Step 2: Write the agent

Replace `agent.py` with:

```
Python
"""
Research-to-Article Pipeline
Demonstrates agent communication via output_key and state templating.

Reference: https://google.github.io/adk-docs/agents/multi-agents#communication
"""

from google.adk.agents import LlmAgent, SequentialAgent

# Step 1: Research agent writes findings to state
researcher = LlmAgent(
    model='gemini-2.5-flash',
    name='researcher',
    output_key='research_findings', # Writes to state
    instruction=''
    Research the given topic thoroughly.

    Provide:
    - 3-5 key facts
    - Important statistics or data
    - Current trends or developments

    Be factual and informative.
    ...
)

# Step 2: Writer reads research and creates article
writer = LlmAgent(
    model='gemini-2.5-flash',
    name='writer',
    output_key='draft_article', # Writes to state
    instruction=''
    Write a short article based on this research:

    {research_findings}

    Structure:
    - Engaging introduction
    - Key findings (2-3 paragraphs)
    - Brief conclusion

    Keep it under 300 words.
    ...
)

# Step 3: Editor reads article and polishes
editor = LlmAgent(
    model='gemini-2.5-flash',
    name='editor',
    instruction=''
    Edit and polish this article:

    {draft_article}

    Fix any grammar issues, improve flow, and ensure clarity.
    Return the final polished version.
    ...
)

# Pipeline: Research → Write → Edit
root_agent = SequentialAgent(
    name='article_pipeline',
    sub_agents=[researcher, writer, editor]
)
```

What this code does:

- 1 Researcher writes findings to `state['research_findings']`
- 2 Writer reads `{research_findings}`, writes article to `state['draft_article']`
- 3 Editor reads `{draft_article}` and produces final version

Step 3: Run and test

```
Shell  
adk web
```

Navigate to <http://localhost:8000>.

Step 4: Test it

You: `Write an article about renewable energy`

Expected flow:

- 1 Researcher gathers facts about renewable energy
- 2 Writer creates article using those facts
- 3 Editor polishes the final article

What to notice:

- Each agent builds on the previous one's work
- Data flows through `output_key` and `{key}` templating
- The pipeline is sequential and predictable





Key takeaways

Agent communication in ADK centers on a shared session state. This design keeps agents loosely coupled while allowing them to work together on complex tasks. The `output_key` and `{key}` templating pattern handles the vast majority of communication needs.

The primary communication pattern:

- Use `output_key` to save agent output to state
- Use `{key}` in instructions to read from state
- Data flows through pipelines via shared state

How it works:

- 1 Agent A sets `output_key='data'` → saves to `state['data']`
- 2 Agent B uses `{data}` in instruction → reads from `state['data']`
- 3 All agents in the hierarchy share the same session state

Other patterns (for reference):

- `sub_agents` with LLM routing for dynamic delegation
- `AgentTool` for explicit tool-like invocation

Remember: Start with `output_key` - it's the essential pattern for agent communication.

